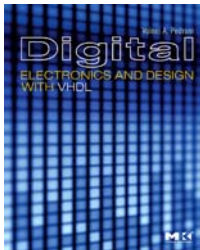


Circuit Design with VHDL

Volnei A. Pedroni
MIT Press – Aug. 2004

Problem Solutions (*)

(*) Details about the physical circuits, codes, and binary algebra can be found in reference [1] below.



[1] V. A. Pedroni, *Digital Electronics and Design with VHDL*, Elsevier, 2008.

Book website: <http://textbooks.elsevier.com/9780123742704>

Chapter 2: Code Structure

Problem 2.1: Multiplexer

Solution:

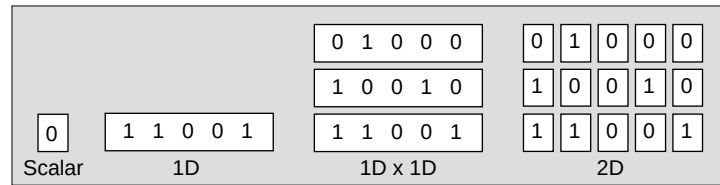
(Physical circuits and operation of multiplexers are described in chapter 11 of [1].)

```
-----  
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
-----  
ENTITY mux IS  
    PORT (a, b: IN STD_LOGIC_VECTOR(7 DOWNTO 0);  
          sel: IN STD_LOGIC_VECTOR(1 DOWNTO 0);  
          c: OUT STD_LOGIC_VECTOR(7 DOWNTO 0));  
END mux;  
-----  
ARCHITECTURE example OF mux IS  
BEGIN  
    PROCESS (a, b, sel)  
    BEGIN  
        IF (sel="00") THEN  
            c <= "00000000";  
        ELSIF (sel="01") THEN  
            c <= a;  
        ELSIF (sel="10") THEN  
            c <= b;  
        ELSE  
            c <= (OTHERS => 'Z'); --or c <= "ZZZZZZZZ";  
        END IF;  
    END PROCESS;  
END example;  
-----
```

Chapter 3: Data Types

Problem 3.2: Dealing with data types

Solution:



Examples of data structures.

First, recall figure 3.1 (repeated above), which shows four types of data structures. From it, we conclude the following for the signals listed in Problem 3.2:

a: a scalar of type BIT.

b: a scalar of type STD_LOGIC.

x: a 1D array (a vector) of type ARRAY1, whose 8 individual elements are of type STD_LOGIC.

y: a 2D array (a matrix) of type ARRAY2, whose 4x8=32 individual elements are of type STD_LOGIC.

w: a 1Dx1D array (another matrix) of type ARRAY3, whose 4 individual 8-element vectors are of type ARRAY1.

z: another 1D array (another vector) whose 8 individual elements are again of type STD_LOGIC.

Therefore:

```
a <= x(2);
```

a: scalar, type BIT; *x*(2): scalar, type STD_LOGIC; assignment is illegal (type mismatch).

```
b <= x(2);
```

b: scalar, type STD_LOGIC; *x*(2): scalar, type STD_LOGIC; assignment is legal.

```
b <= y(3,5);
```

b: scalar, type STD_LOGIC; *y*(3,5): scalar, type STD_LOGIC, with valid indexing; assignment is legal.

```
b <= w(5)(3);
```

b: scalar, type STD_LOGIC; *w*(5,3): scalar, type STD_LOGIC, but “5” is out of bounds; assignment is illegal.

```
y(1)(0) <= z(7);
```

y(1)(0): scalar, type STD_LOGIC, but indexing is incorrect because *y* is 2D (it should be *y*(1,0)); *z*(7): scalar, type STD_LOGIC; assignment is illegal.

```
x(0) <= y(0,0);
```

x(0): scalar, type STD_LOGIC; *y*(0,0): scalar, type STD_LOGIC, valid indexing; assignment is legal.

```
x <= "11100000";
```

x: 8-bit vector (1D); assignment would be legal if it contained 8 values instead of 7.

```
a <= "00000000";
```

a is scalar (one bit), so the assignment is illegal.

```
y(1) <= x;
```

y(1): in principle, an 8-element vector, extracted from a 2D matrix, whose individual elements are of type STD_LOGIC; however, the indexing of *y* is not valid, because the matrix is 2D, not 1Dx1D; *x*: an 8-element vector of type ARRAY1; assignment is illegal (invalid indexing + type mismatch).

```
w(0) <= y;
```

w(0): row 0 of a 1Dx1D matrix, which is an 8-element vector of type ARRAY1; *y*: a 4x8 (2D) matrix; assignment is illegal (size + type mismatches).

```
w(1) <= (7=>'1', OTHERS=>'0');
```

w(1): row 1 of a 1Dx1D matrix; assignment w(1)<="10000000" is legal.

```
y(1) <= (0=>'0', OTHERS=>'1');
```

y(1): in principle, row 1 of a matrix, but the indexing is invalid, because the matrix is 2D, not 1Dx1D; assignment y(1)<="11111110" is illegal.

```
w(2)(7 DOWNT0 0) <= x;
```

w(2)(7 DOWNT0 0): row 2 of a 1Dx1D matrix, which is an 8-element vector of type ARRAY1; x: an 8-element vector of type ARRAY1; assignment is legal.

Note: w(2)<=x would be fine too.

```
w(0)(7 DOWNT0 6) <= z(5 DOWNT0 4);
```

w(0)(7 DOWNT0 6): the leftmost 2 elements of row 0 of a 1Dx1D matrix, being each row an 8-element vector of type ARRAY1; z(5 DOWNT0 4): 2 elements of a 8-element STD_LOGIC_VECTOR; assignment is illegal (type mismatch).

```
x(3) <= x(5 DOWNT0 5);
```

x(3): a scalar of type STD_LOGIC; x(5 DOWNT0 5): also a scalar of type STD_LOGIC; assignment is legal.

```
b <= x(5 DOWNT0 5)
```

b: a scalar of type STD_LOGIC; x(5 DOWNT0 5): also a scalar of type STD_LOGIC; assignment is legal.

```
y <= ((OTHERS=>'0'), (OTHERS=>'0'), (OTHERS=>'0'), "10000001");
```

y is a 2D matrix; assignment is legal.

Note: Since y is 2D, some older compilers might not accept the vector-like assignments above, thus requiring the assignment to be made element by element (with GENERATE, for example).

The assignment below is legal too.

```
y <= (('0','0','0','0','0','0','0','0'),
      ('0','0','0','0','0','0','0','0'),
      ('0','0','0','0','0','0','0','0'),
      ('0','0','0','0','0','0','0','0'));
```

```
z(6) <= x(5);
```

z(6): scalar of type STD_LOGIC; x(5): also a scalar of type STD_LOGIC; assignment is legal. Notice that though x, as a vector, is of type ARRAY1, as a scalar the “base” is STD_LOGIC.

```
z(6 DOWNT0 4) <= x(5 DOWNT0 3);
```

z(6 DOWNT0 4): 3-element vector of type STD_LOGIC_VECTOR; x(5 DOWNT0 3): 3-element vector of type ARRAY1; assignment is illegal (type mismatch).

```
z(6 DOWNT0 4) <= y(5 DOWNT0 3);
```

The indexing of y is invalid (and the dimensions of the two spaces do not match); assignment is illegal.

```
y(6 DOWNT0 4) <= x(3 TO 5);
```

Same as above; assignment is illegal.

```
y(0, 7 DOWNT0 0) <= z;
```

y(0, 7 DOWNT0 0): in principle, row 0 of a matrix, but slicing 2D arrays is generally not supported; assignment is illegal.

```
w(2,2) <= '1';
```

w is 1Dx1D, so indexing should w(2)(2); assignment is illegal.

Problem 3.4: ROM implementation

Solution:

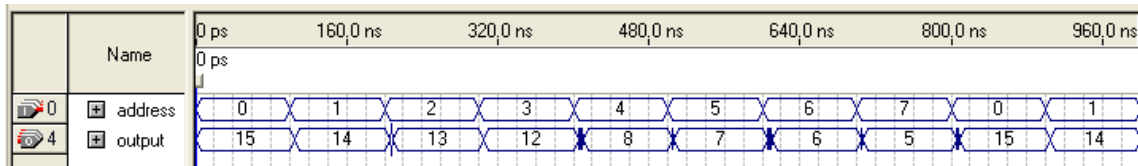
(Physical circuits and operation of ROM memories are described in chapter 17 of [1].)

Simulation results are shown after the code.

```

-----
ENTITY rom_design IS
  PORT (address: IN INTEGER RANGE 0 TO 7;
        output: OUT BIT_VECTOR(3 DOWNTO 0));
END rom_design;
-----
ARCHITECTURE rom_design OF rom_design IS
  TYPE rom IS ARRAY (0 TO 7) OF BIT_VECTOR(3 DOWNTO 0);
  CONSTANT my_rom: ROM := ("1111", --address=0, value=15
                           "1110", --address=1, value=14
                           "1101", --address=2, value=13
                           "1100", --address=3, value=12
                           "1000", --address=4, value=8
                           "0111", --address=5, value=7
                           "0110", --address=6, value=6
                           "0101");--address=7, value=5
BEGIN
  --either of the two lines below is fine:
  output <= my_rom(address);
  --output <= my_rom(address)(3 DOWNTO 0);
END rom_design;
-----

```



Simulation results for Problem 3.4.

Chapter 4: Operators and Attributes

Problem 4.1: Operators

Solution:

```

x1 <= a & c; → x1 <= "10010";
x2 <= c & b; → x2 <= "00101100";
x3 <= b XOR c; → x3 <= "1110";
x4 <= a NOR b(3); → x4 <= '0';
x5 <= b sll 2; → x5 <= "0000";
x6 <= b sla 2; → x6 <= "1000";
x7 <= b rol 2; → x7 <= "0011";
x8 <= a AND NOT b(0) AND NOT c(1); → x8 <= '0';
d <= (5=>'0', OTHERS=>'1'); → d <= "11011111";

```

Problem 4.4: Generic decoder

Solution:

(Physical circuits and operation of decoders are described in chapter 11 of [1].)

```

-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY decoder IS
  GENERIC (n: INTEGER := 8); --number of bits
  PORT (ena: IN STD_LOGIC;
        sel: IN INTEGER RANGE 0 TO n-1;

```

```

        x: OUT STD_LOGIC_VECTOR(n-1 DOWNT0 0));
END decoder;
-----
ARCHITECTURE decoder OF decoder IS
BEGIN
    PROCESS (ena, sel)
        VARIABLE temp : STD_LOGIC_VECTOR (x'RANGE);
    BEGIN
        temp := (OTHERS => '1');
        IF (ena='1') THEN
            temp(sel) := '0';
        END IF;
        x <= temp;
    END PROCESS;
END decoder;
-----

```

Chapter 5: Concurrent Code

Problem 5.1: Generic multiplexer

Solution 1: For generic n and fixed m

(Physical circuits and operation of multiplexers are described in chapter 11 of [1].)

In this solution, a PACKAGE, called *my_data_types*, is employed to define a new data type, called *vector_array*. This data type is then used in the ENTITY of the main code to specify the inputs of the multiplexer. Notice in the main code that the number of inputs is defined using the GENERIC statement.

```

-----Package:-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
PACKAGE my_data_types IS
    CONSTANT m: INTEGER :=8;
    TYPE vector_array IS ARRAY (NATURAL RANGE <>) OF
        STD_LOGIC_VECTOR(m-1 DOWNT0 0);
END my_data_types;
-----

-----Main code:-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE work.my_data_types.all;
-----
ENTITY almost_generic_mux IS
    GENERIC (number_of_inputs: INTEGER := 16);
    PORT (x: IN VECTOR_ARRAY (0 TO number_of_inputs-1);
          sel: IN INTEGER RANGE 0 TO number_of_inputs-1;
          y: OUT STD_LOGIC_VECTOR (m-1 DOWNT0 0));
END almost_generic_mux;
-----
ARCHITECTURE mux OF almost_generic_mux IS
BEGIN
    y <= x(sel);
END mux;
-----

```

Solution 2: For n and m generic

```

-----Package:-----
PACKAGE my_data_types IS
    TYPE matrix IS ARRAY (NATURAL RANGE <>, NATURAL RANGE <>) OF BIT;
END PACKAGE my_data_types;
-----

```

```

-----Main code:-----
USE work.my_data_types.all;
-----
ENTITY generic_mux IS
    GENERIC (inputs: INTEGER := 16; --number of inputs
             size: INTEGER := 8); --size of each input
    PORT (x: IN MATRIX (0 TO inputs-1, size-1 DOWNT0 0);
          sel: IN INTEGER RANGE 0 TO inputs-1;
          y: OUT BIT_VECTOR (size-1 DOWNT0 0));
END generic_mux;
-----
ARCHITECTURE mux OF generic_mux IS
BEGIN
    gen: FOR i IN size-1 DOWNT0 0 GENERATE
        y(i) <= x(sel, i);
    END GENERATE gen;
END mux;
-----

```

Problem 5.2: Priority encoder

Solution for part (a):

```

-----
ENTITY priority_encoder IS
    PORT (x: IN BIT_VECTOR(7 DOWNT0 1);
          y: OUT BIT_VECTOR(2 DOWNT0 0));
END priority_encoder;
-----
ARCHITECTURE encoder OF priority_encoder IS
BEGIN
    y(2) <= x(7) OR x(6) OR x(5) OR x(4);
    y(1) <= x(7) OR x(6) OR ((NOT x(5) AND NOT x(4)) AND
                             (x(3) OR x(2)));
    y(0) <= x(7) OR (NOT x(6) AND (x(5) OR (NOT x(4) AND
                             (x(3) OR (NOT x(2) AND x(1))))));
END encoder;
-----

```

Solution for part (b):

```

-----
ENTITY priority_encoder IS
    PORT (x: IN BIT_VECTOR(7 DOWNT0 1);
          y: OUT BIT_VECTOR(2 DOWNT0 0));
END priority_encoder;
-----
ARCHITECTURE encoder OF priority_encoder IS
BEGIN
    y <= "111" WHEN x(7)='1' ELSE
         "110" WHEN x(6)='1' ELSE
         "101" WHEN x(5)='1' ELSE
         "100" WHEN x(4)='1' ELSE
         "011" WHEN x(3)='1' ELSE
         "010" WHEN x(2)='1' ELSE
         "001" WHEN x(1)='1' ELSE
         "000";
END encoder;
-----

```

Note: See a *generic* implementation in the solution of Problem 6.3.

Problem 5.4: Unsigned adder

Solution:

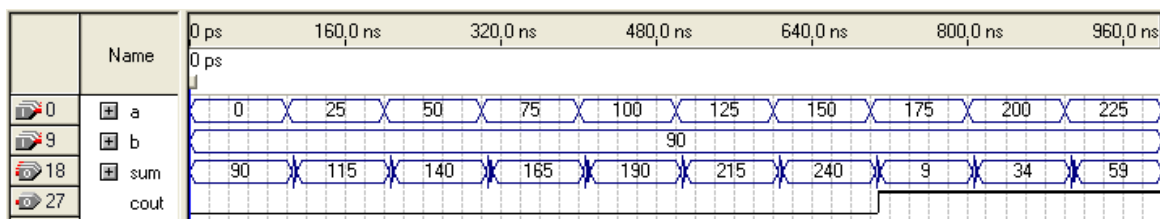
(Physical circuits and operation of adders are described in chapters 3 and 12 of [1].)

In the solution below, the inputs and outputs are considered to be of type STD_LOGIC (industry standard). Simulation results are included after the code.

```

-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_signed.all; --allows arith operations w/ STD_LOGIC
-----
ENTITY adder IS
    PORT (a, b: IN STD_LOGIC_VECTOR(7 DOWNTO 0);
          sum: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
          cout: OUT STD_LOGIC);
END adder;
-----
ARCHITECTURE adder OF adder IS
    SIGNAL long_a, long_b, long_sum: STD_LOGIC_VECTOR(8 DOWNTO 0);
BEGIN
    long_a <= '0' & a;
    long_b <= '0' & b;
    long_sum <= long_a + long_b;
    sum <= long_sum(7 DOWNTO 0);
    cout <= long_sum(8);
END adder;
-----

```



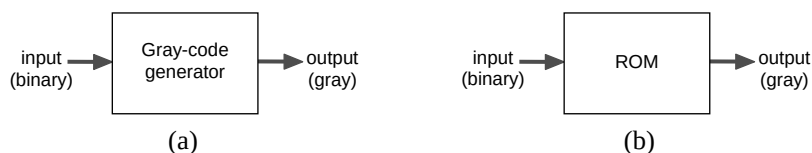
Simulation results for Problem 5.4.

Problem 5.6: Binary-to-Gray code converter

Solution:

(Gray code is described in chapter 2 of [1].)

The figure below shows two approaches to solve this problem. In (a), a circuit is used to generate the Gray code corresponding to the (conventional) binary word presented at its input. In (b), a ROM (Read-Only Memory), similar to that seen in Problem 3.4, is used, in which the input acts as the address and the output is the ROM's content at that address.



Two approaches to attain a binary-to-Gray converter.

Since the problem asks for a *generic* solution (that is, for any number of bits, N), approach (a) will be employed. The Gray word, $g(N-1:0)$, corresponding to a regular binary word, $b(N-1:0)$, can be obtained in several ways, among them the following:

$$g(N-1:0) = b(N-1:0) \text{ XOR } (b(N-1:0) \text{ SRL } 1)$$

Or, equivalently:

$$g(N-1:0) = b(N-1:0) \text{ XOR } ('0' \& b(N-1:1))$$

Or yet:

$$g(N-1) = b(N-1)$$

$$g(N-2:0) = b(N-2:0) \text{ XOR } b(N-1:1)$$

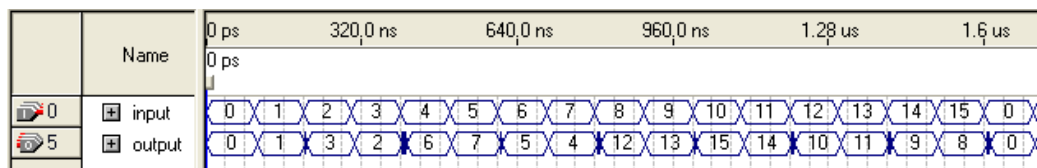
A corresponding, *generic* VHDL code is presented below, followed by simulation results for $N=4$.

Note: See the question for the reader at the end of this exercise.

```

-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY gray_encoder IS
    GENERIC (N: INTEGER := 4);
    PORT (input: IN STD_LOGIC_VECTOR(N-1 DOWNTO 0);
          output: OUT STD_LOGIC_VECTOR(N-1 DOWNTO 0));
END gray_encoder;
-----
ARCHITECTURE gray_encoder OF gray_encoder IS
BEGIN
    output(N-1) <= input(N-1);
    output(N-2 DOWNTO 0) <= input(N-2 DOWNTO 0) XOR input(N-1 DOWNTO 1);
END gray_encoder;
-----

```



Simulation results for Problem 5.6.

Question regarding the solution above: Three equivalent equations were presented to calculate Gray words from (regular) binary words. However, from a VHDL perspective, the first equation (that with the shift-right logic operator, SRL) is inferior to the others. Explain why. (Suggestion: Check in the package *standard* which data types are supported by the shift operators.)

Problem 5.8: Signed/unsigned comparator

Solution:

(Physical circuits and operation of comparators are described in chapter 12 of [1].)

In the code below, the package *std_logic_arith* was used because it contains the conversion function CONV_SIGNED as well as respective comparison operators. Note in the simulation results that the numbers are always non-negative while *sel*='0', but are signed for *sel*='1' (so, for example, 150 is indeed -106). The glitches at some of the state transitions are absolutely normal.

```

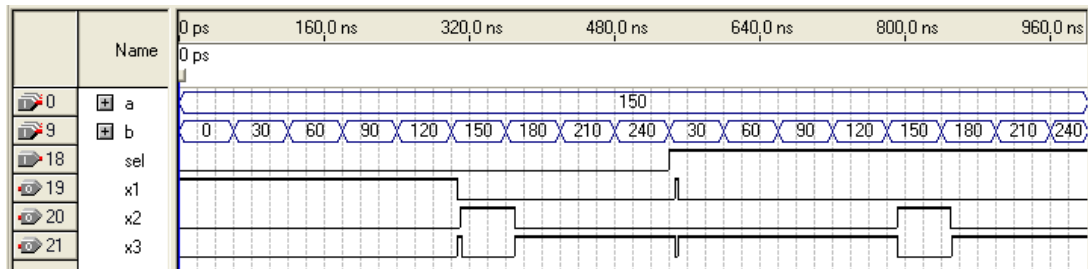
-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_arith.all;
-----
ENTITY comparator IS
    PORT (a, b: IN INTEGER RANGE 0 TO 255;
          sel: IN STD_LOGIC;
          x1, x2, x3: OUT STD_LOGIC);
END comparator;
-----
ARCHITECTURE comparator OF comparator IS

```

```

    SIGNAL a_signed, b_signed: SIGNED(7 DOWNTO 0);
    SIGNAL temp: STD_LOGIC_VECTOR(1 TO 4);
BEGIN
    a_signed <= CONV_SIGNED(a, 8);
    b_signed <= CONV_SIGNED(b, 8);
    temp(1) <= '1' WHEN a>b ELSE
        '0';
    temp(2) <= '1' WHEN a=b ELSE
        '0';
    temp(3) <= '1' WHEN a_signed>b_signed ELSE
        '0';
    temp(4) <= '1' WHEN a_signed=b_signed ELSE
        '0';
    x1 <= temp(1) WHEN sel='0' ELSE
        temp(3);
    x2 <= temp(2) WHEN sel='0' ELSE
        temp(4);
    x3 <= NOT(temp(1) OR temp(2)) WHEN sel='0' ELSE
        NOT(temp(2) OR temp(3));
END comparator;
-----

```



Simulation results for Problem 5.8.

Chapter 6: Sequential Code

Problem 6.3: Priority encoder

Solution 1: Non-generic

Two solutions are presented for part(a). Solution 1 is explicit (“brute force”), while solution 2 is generic. The latter is obviously recommended. Simulation results are also included.

```

-----
ENTITY priority_encoder IS
    PORT (x: IN BIT_VECTOR(7 DOWNTO 1);
          y: OUT BIT_VECTOR(2 DOWNTO 0));
END priority_encoder;
-----
ARCHITECTURE encoder OF priority_encoder IS
BEGIN
    PROCESS(x)
    BEGIN
        IF (x(7)='1') THEN
            y <= "111";
        ELSIF (x(6)='1') THEN
            y <= "110";
        ELSIF (x(5)='1') THEN
            y <= "101";
        ELSIF (x(4)='1') THEN
            y <= "100";
        ELSIF (x(3)='1') THEN
            y <= "011";
        ELSIF (x(2)='1') THEN

```

```

        y <= "010";
    ELSIF (x(1)='1') THEN
        y <= "001";
    ELSE
        y <= "000";
    END IF;
END PROCESS;
END encoder;
-----

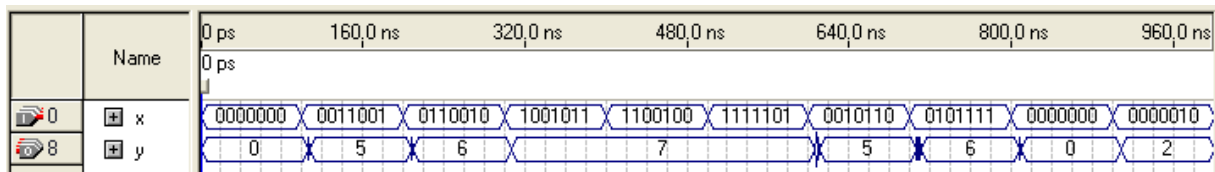
```

Solution 2: Generic

```

-----
ENTITY priority_encoder IS
    GENERIC (N: INTEGER := 3); --number of output bits
    PORT (x: IN BIT_VECTOR(2**N-1 DOWNT0 1);
          y: OUT INTEGER RANGE 0 TO 2**N-1);
END priority_encoder;
-----
ARCHITECTURE priority_encoder OF priority_encoder IS
BEGIN
    PROCESS(x)
        VARIABLE temp: INTEGER RANGE 0 TO 2**N-1;
    BEGIN
        temp := 0;
        FOR i IN x'RANGE LOOP
            IF (x(i)='1') THEN
                temp := i;
                EXIT;
            END IF;
        END LOOP;
        y <= temp;
    END PROCESS;
END priority_encoder;
-----

```



Simulation results for Problem 6.3.

Problem 6.4: Generic frequency divider

Solution:

(Physical circuits and operation of frequency dividers are described in chapters 14 and 15 of [1].)

```

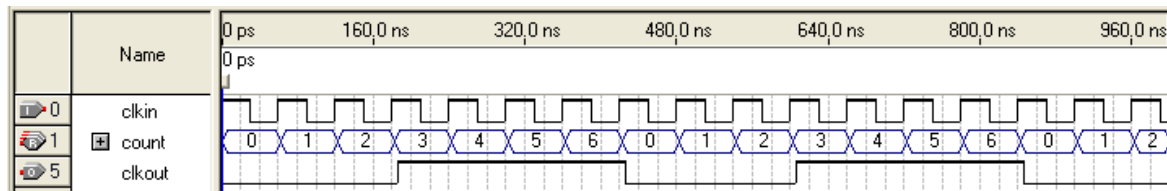
----Clock frequency is divided by n-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY clock_divider IS
    GENERIC (n: INTEGER := 7);
    PORT (clk_in: IN STD_LOGIC;
          clk_out: OUT STD_LOGIC);
END clock_divider;
-----
ARCHITECTURE clock_divider OF clock_divider IS
BEGIN
    PROCESS (clk_in)
        VARIABLE count: INTEGER RANGE 0 TO n;
    BEGIN
        IF (clk_in'EVENT AND clk_in='1') THEN

```

```

        count := count + 1;
        IF (count=n/2) THEN
            clkout <= '1';
        ELSIF (count=n) THEN
            clkout <= '0';
            count := 0;
        END IF;
    END IF;
END PROCESS;
END clock_divider;
-----

```



Simulation results for Problem 6.4.

Problem 6.6: Timer #1

Solution:

(Physical circuits and operation of timers are described in chapter 14 of [1].)

In the code below, the problem was modified slightly:

- The *start* and *stop* inputs were replaced with a single control signal, called *run*, which causes the circuit to count when *run*='1' (given that 9:59 has not been reached yet) or keep its last state when *run*='0'.
- The clock frequency was entered using the GENERIC attribute, so the circuit can be easily adjusted to any clock frequency.

The SSDs (seven-segment displays) were considered to be *common cathode* (positive logic), with the segments represented by *abcdefgx*, where *x* is the SSD's decimal point (kept off in the digits of seconds and on in the digit of minutes).

A little portion of the experimental results is included after the code below.

```

-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY timer1 IS
    GENERIC (fclk: INTEGER := 5); --clock frequency in Hz
    PORT (clk, rst, run: IN STD_LOGIC;
          digit1, digit2, digit3: OUT STD_LOGIC_VECTOR(7 DOWNTO 0));
END timer1;
-----
ARCHITECTURE timer1 OF timer1 IS
    SIGNAL oneHz_clk: STD_LOGIC;
    SIGNAL seconds1: INTEGER RANGE 0 TO 10; --units of seconds
    SIGNAL seconds2: INTEGER RANGE 0 TO 6;  --tens of seconds
    SIGNAL minutes: INTEGER RANGE 0 TO 10;  --units of minutes
BEGIN
    ---1Hz clock:-----
    PROCESS (clk)
        VARIABLE count: INTEGER RANGE 0 TO fclk;
    BEGIN
        IF (clk'EVENT AND clk='1') THEN
            count := count + 1;
            IF (count=fclk/2) THEN
                oneHz_clk <= '1';
            ELSIF (count=fclk) THEN
                oneHz_clk <= '0';
            END IF;
        END IF;
    END PROCESS;

```

```

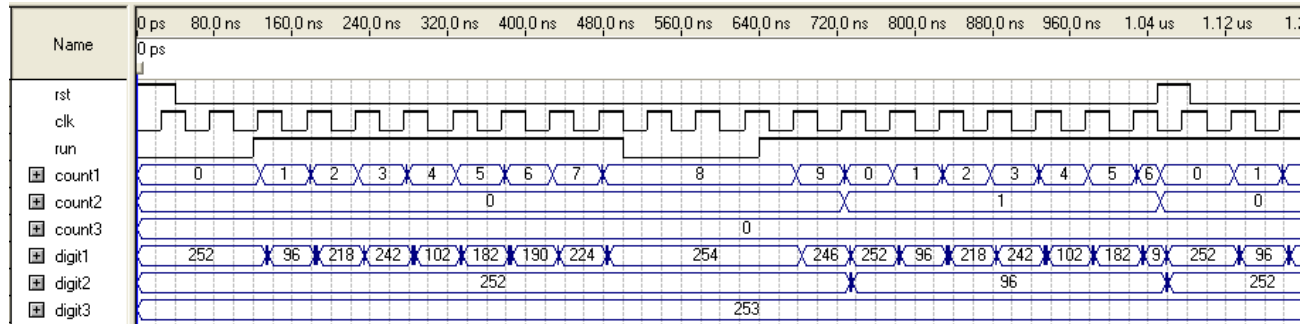
        count := 0;
    END IF;
END IF;
END PROCESS;
----BCD counters:-----
PROCESS (oneHz_clk, rst)
    VARIABLE count1: INTEGER RANGE 0 TO 10; --units of seconds
    VARIABLE count2: INTEGER RANGE 0 TO 6;  --tens of seconds
    VARIABLE count3: INTEGER RANGE 0 TO 10; --units of minutes
BEGIN
    IF (rst='1') THEN
        count1:=0;
        count2:=0;
        count3:=0;
    ELSIF (clk'EVENT AND clk='1') THEN
        IF (run='1' AND (count1/=9 OR count2/=5 OR count3/=9)) THEN
            count1 := count1 + 1;
            IF (count1=10) THEN
                count1 := 0;
                count2 := count2 + 1;
                IF (count2=6) THEN
                    count2 := 0;
                    count3 := count3 + 1;
                END IF;
            END IF;
        END IF;
    END IF;
    seconds1 <= count1;
    seconds2 <= count2;
    minutes <= count3;
END PROCESS;
---Conversion from BCD to SSD:-----
PROCESS (seconds1, seconds2, minutes)
BEGIN
    CASE seconds1 IS
        WHEN 0 => digit1 <= "11111100"; --decimal 252
        WHEN 1 => digit1 <= "01100000"; --decimal 96
        WHEN 2 => digit1 <= "11011010"; --decimal 218
        WHEN 3 => digit1 <= "11110010"; --decimal 242
        WHEN 4 => digit1 <= "01100110"; --decimal 102
        WHEN 5 => digit1 <= "10110110"; --decimal 182
        WHEN 6 => digit1 <= "10111110"; --decimal 190
        WHEN 7 => digit1 <= "11100000"; --decimal 224
        WHEN 8 => digit1 <= "11111110"; --decimal 254
        WHEN 9 => digit1 <= "11110110"; --decimal 246
        WHEN OTHERS => NULL;
    END CASE;
    CASE seconds2 IS
        WHEN 0 => digit2 <= "11111100"; --decimal 252
        WHEN 1 => digit2 <= "01100000"; --decimal 96
        WHEN 2 => digit2 <= "11011010"; --decimal 218
        WHEN 3 => digit2 <= "11110010"; --decimal 242
        WHEN 4 => digit2 <= "01100110"; --decimal 102
        WHEN 5 => digit2 <= "10110110"; --decimal 182
        WHEN OTHERS => NULL;
    END CASE;
    CASE minutes IS
        WHEN 0 => digit3 <= "11111101"; --decimal 253
        WHEN 1 => digit3 <= "01100001"; --decimal 97
        WHEN 2 => digit3 <= "11011011"; --decimal 219
        WHEN 3 => digit3 <= "11110011"; --decimal 243
        WHEN 4 => digit3 <= "01100111"; --decimal 103
        WHEN 5 => digit3 <= "10110111"; --decimal 183
        WHEN 6 => digit3 <= "10111111"; --decimal 192
        WHEN 7 => digit3 <= "11100001"; --decimal 225
        WHEN 8 => digit3 <= "11111111"; --decimal 255
    END CASE;
END PROCESS;

```

```

        WHEN 9 => digit3 <= "11110111"; --decimal 247
        WHEN OTHERS => NULL;
    END CASE;
END PROCESS;
END timer1;

```



Simulation results for Problem 6.6.

Problem 6.8: Parity detector

Solution:

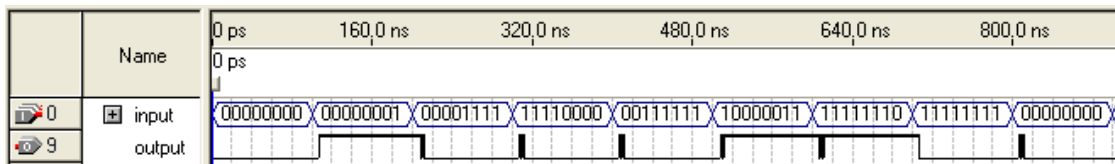
(Physical circuits and operation of parity detectors are described in chapter 11 of [1].)

The solution below, using sequential code, is fine for any number of bits (N). Simulation results are also presented.

```

-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY parity_detector IS
    GENERIC (N: INTEGER := 8); --number of bits
    PORT (input: IN STD_LOGIC_VECTOR(N-1 DOWNT0 0);
          output: OUT STD_LOGIC);
END parity_detector;
-----
ARCHITECTURE parity_detector OF parity_detector IS
BEGIN
    PROCESS (input)
        VARIABLE temp: STD_LOGIC_VECTOR(N-1 DOWNT0 0);
    BEGIN
        temp(0) := input(0);
        FOR i IN 1 TO N-1 LOOP
            temp(i) := input(i) XOR temp(i-1);
        END LOOP;
        output <= temp(N-1);
    END PROCESS;
END parity_detector;
-----

```



Simulation results for Problem 6.8.

Problem 6.16: Carry-ripple adder

Solution:

(Physical circuits and operation of adders are described in chapters 3 and 12 of [1].)

Part (a): Because of type mismatch (*temp* is INTEGER, *c0* is STD_LOGIC).

Part (b): See the code below.

```
-----  
LIBRARY ieee;  
USE ieee.std_logic_1164.all;  
-----  
ENTITY carry_ripple_adder IS  
    GENERIC (length: INTEGER := 8);  
    PORT (a, b: IN STD_LOGIC_VECTOR(length-1 DOWNT0 0);  
          cin: IN STD_LOGIC;  
          s: OUT STD_LOGIC_VECTOR(length-1 DOWNT0 0);  
          cout: OUT STD_LOGIC);  
END carry_ripple_adder;  
-----  
ARCHITECTURE adder OF carry_ripple_adder IS  
    SIGNAL carry: STD_LOGIC_VECTOR(length DOWNT0 0);  
BEGIN  
    carry(0) <= cin;  
    generate_output: FOR i IN 0 TO length-1 GENERATE  
        s(i) <= a(i) XOR b(i) XOR carry(i);  
        carry(i+1) := (a(i) AND b(i)) OR (a(i) AND  
            carry(i)) OR (b(i) AND carry(i));  
    END GENERATE;  
    cout <= carry(length);  
END adder;  
-----
```

Problem 6.17: DFF

Solution:

(Physical circuits and operation of all types of flip-flops are described in chapter 13 of [1].)

Architecture 1: A truly D-type flip-flop with asynchronous reset (same as in Example 6.1) .

Architecture 2: Only the clock appears in the sensitivity list, causing the reset to be *synchronous*.

Architecture 3: clk'EVENT is not associated with a test (AND clk='1', for example). Some compilers might assume a default test (AND clk='1'), while others will simply halt. Consequently, to avoid mistakes, explicitly writing the test is advisable.

Architecture 4: Here the contents of sections 6.10 and 7.5 are helpful. Notice that no signal assignment is made at the transition of another signal, signifying that, in principle, no flip-flop is wanted (probably a combinational circuit). However, only one of the input signals appears in the sensitivity list, and an incomplete truth table for *q* is specified in the code, causing the inference of a latch to hold *q*'s value (a "pseudo" combinational circuit).

Architecture 5: The situation here is even more awkward than that above. Besides generating again a latch, the value of *d* also causes the process to be run. The result is a circuit very unlikely to be of any interest.

Chapter 7: Signals and Variable

Problem 7.2: Data delay

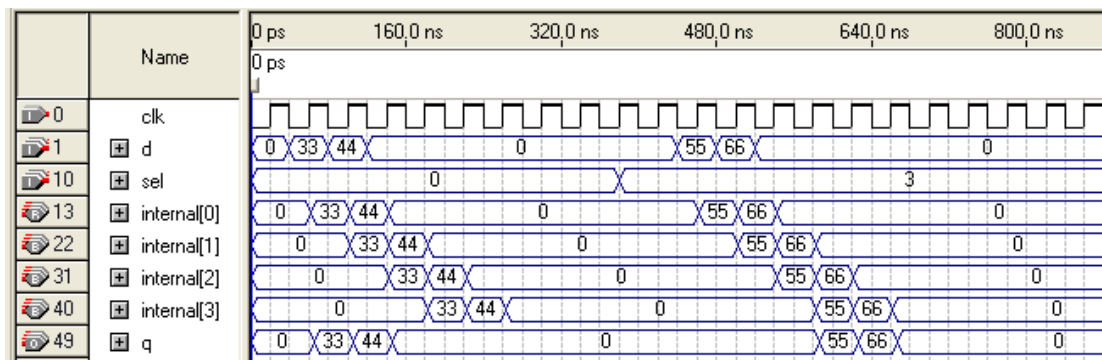
Solution:

(Physical circuits and operation of data delays using shift registers are described in chapter 14 of [1].)

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY data_delay IS
    PORT (d: IN STD_LOGIC_VECTOR(7 DOWNTO 0);
          clk: IN STD_LOGIC;
          sel: IN INTEGER RANGE 0 TO 3;
          q: OUT STD_LOGIC_VECTOR(7 DOWNTO 0));
END data_delay;
-----
ARCHITECTURE data_delay OF data_delay IS
    TYPE register_array IS ARRAY (0 TO 3) OF
        STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL internal: register_array;
BEGIN
    PROCESS (clk, sel)
    BEGIN
        IF (clk'EVENT AND clk='1') THEN
            internal <= d & internal(0 TO 2);
        END IF;
    END PROCESS;
    q <= internal(sel);
END data_delay;
-----

```



Simulation results for Problem 7.2.

Problem 7.6: Generic address decoder**Solution:**

(Physical circuits and operation of decoders are described in chapter 11 of [1].)

```

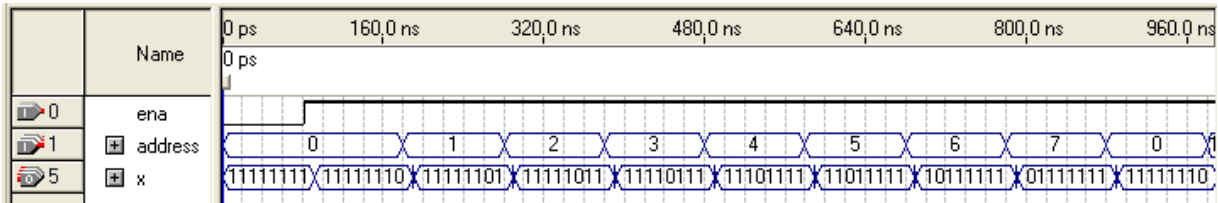
-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY address_decoder IS
    GENERIC (N: INTEGER := 8); --#number of output bits
    PORT (ena: IN STD_LOGIC;
          address: IN INTEGER RANGE 0 TO N-1;
          x: OUT STD_LOGIC_VECTOR(N-1 DOWNTO 0));
END address_decoder;
-----
ARCHITECTURE behavior OF address_decoder IS
BEGIN
    PROCESS (ena, address)
        VARIABLE temp: STD_LOGIC_VECTOR(x'RANGE);
    BEGIN
        temp := (OTHERS => '1');
        IF (ena='1') THEN
            temp(address) := '0';
        END IF;
    END PROCESS;
END behavior;

```

```

    x <= temp;
END PROCESS;
END behavior;

```



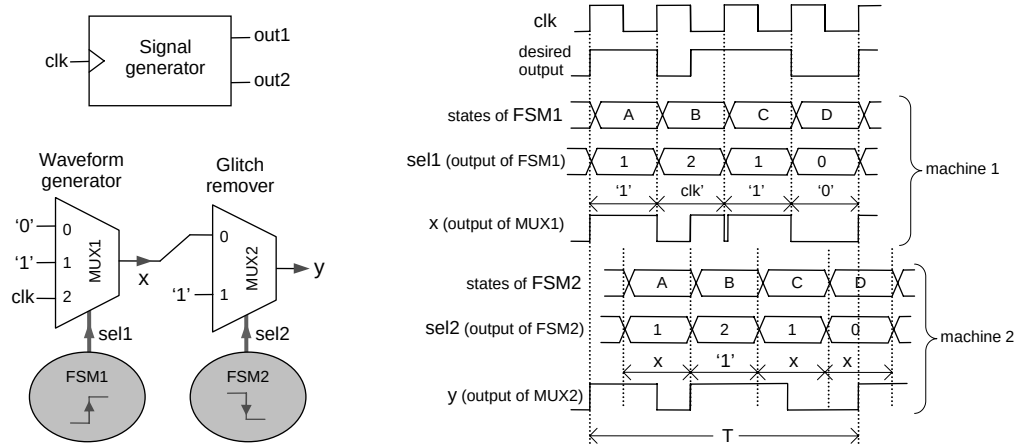
Simulation results for Problem 7.6.

Chapter 8: State Machines

Problem 8.2: Signal generator #1

Solution:

(Physical circuits, design, and operation of finite state machines are described in chapter 15 of [1]. The solution that follows is based on the *arbitrary signal generator design technique* introduced there. The corresponding signals are depicted in the figure below.)



Details regarding the states machines of Problem 8.2.

In this exercise, *out1* is very simple to generate, because all of its transitions are at the same clock edge (positive). However, the same is not true for *out2*, which contains transitions at both clock edges. Consequently, for the latter, the technique introduced in [1] is very helpful (indeed, the “weirder” the signal to be generated, the more useful that approach is). Coincidentally, the shape of *out2* in this exercise is not subject to glitches during signal construction, so to better illustrate the use of such that technique a slight modification will be introduced (see the waveform right below *clk* in the figure above). Observe that now *x* is subject to glitches, making the problem a more general case.

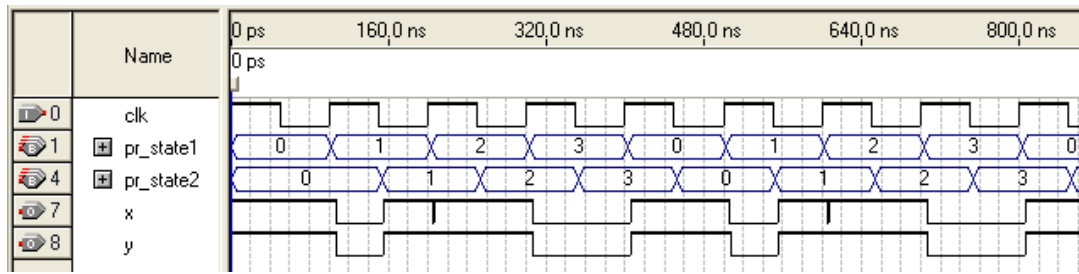
Two FSMs are employed to create the desired signal, each associated with a multiplexer. The pair FSM1-MUX1 is responsible for *generating* the signal (possibly with glitches), while the pair FSM2-MUX2 is responsible for *cleaning* it (recall that in a signal generator glitches are never acceptable). As shown in the figure, FSM1 operates at the positive clock edge, while FSM2 operates at the negative transition. The states (for both machines) are called A, B, C, and D.

A corresponding VHDL code is shown below, followed by simulation results. Observe in the latter the expected glitches in *x*, which disappear in *y*.

```

-----
ENTITY signal_generator IS
    PORT (clk: IN BIT;
          x: BUFFER BIT;
          y: OUT BIT);
END signal_generator;
-----
ARCHITECTURE signal_generator OF signal_generator IS
    TYPE state IS (A, B, C, D);
    SIGNAL pr_state1, nx_state1: state;
    SIGNAL pr_state2, nx_state2: state;
    ATTRIBUTE enum_encoding: STRING;
    ATTRIBUTE enum_encoding OF state: TYPE IS "sequential";
BEGIN
    ----- Lower section of FSM1: -----
    PROCESS (clk)
    BEGIN
        IF (clk'EVENT AND clk='1') THEN
            pr_state1 <= nx_state1;
        END IF;
    END PROCESS;
    ----- Upper section of FSM1: -----
    PROCESS (pr_state1, clk)
    BEGIN
        CASE pr_state1 IS
            WHEN A =>
                x <= '1';
                nx_state1 <= B;
            WHEN B =>
                x <= NOT clk;
                nx_state1 <= C;
            WHEN C =>
                x <= '1';
                nx_state1 <= D;
            WHEN D =>
                x <= '0';
                nx_state1 <= A;
        END CASE;
    END PROCESS;
    ----- Lower section of FSM2: -----
    PROCESS (clk)
    BEGIN
        IF (clk'EVENT AND clk='0') THEN
            pr_state2 <= nx_state2;
        END IF;
    END PROCESS;
    ----- Upper section of FSM2: -----
    PROCESS (pr_state1, pr_state2, x)
    BEGIN
        nx_state2 <= pr_state1; --synchronism
        CASE pr_state2 IS
            WHEN A =>
                y <= x;
            WHEN B =>
                y <= '1';
            WHEN C =>
                y <= x;
            WHEN D =>
                y <= x;
        END CASE;
    END PROCESS;
END signal_generator;
-----

```

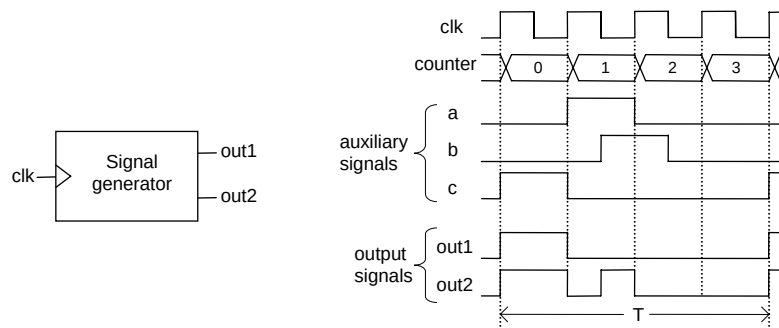


Simulation results for Problem 8.2.

Problem 8.6: Signal generator of Problem 8.2 without FSM approach

Solution:

(Physical circuits, design, and operation of sequential circuits are described in chapter 14 of [1]. The solution that follows is based on the theory presented there. The corresponding signals are depicted in the figure below.)



Signals employed in the solution of Problem 8.6.

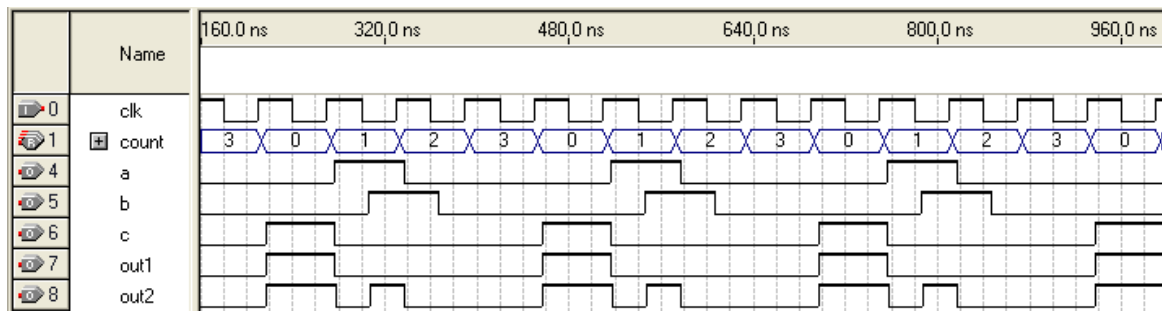
In this case (see figure above), three auxiliary signals (*a*, *b*, *c*) are created, from which *out1* and *out2* are then derived using conventional gates. As described in chapter 14 of [1], the fundamental point here is to guarantee that the outputs are not prone to glitches. To guarantee glitch-free outputs, all three auxiliary signals are registered (i.e., stored in DFFs); consequently, given that no two signals that affect the gates' outputs change at the same clock edge, glitch-free outputs are automatically generated. A corresponding VHDL follows, along with simulation results.

```
-----
ENTITY signal_generator IS
    PORT (clk: IN BIT;
          out1, out2: OUT BIT);
END signal_generator;
-----
ARCHITECTURE signal_generator OF signal_generator IS
    SIGNAL a, b, c: BIT;
    SIGNAL counter: INTEGER RANGE 0 TO 3;
BEGIN
    ----Creating a counter:-----
    PROCESS (clk)
        VARIABLE count: INTEGER RANGE 0 TO 4;
    BEGIN
        IF (clk'EVENT AND clk='1') THEN
            count := count + 1;
            IF (count=4) THEN
                count := 0;
            END IF;
        END IF;
        counter <= count;
    END PROCESS;
    ----Generating signal a:-----
    PROCESS (clk)
```

```

BEGIN
  IF (clk'EVENT AND clk='1') THEN
    IF (counter=0) THEN
      a <= '1';
    ELSE
      a <= '0';
    END IF;
  END IF;
END PROCESS;
----Generating signal b:-----
PROCESS (clk)
BEGIN
  IF (clk'EVENT AND clk='0') THEN
    IF (counter=1) THEN
      b <= '1';
    ELSE
      b <= '0';
    END IF;
  END IF;
END PROCESS;
----Generating signal c:-----
PROCESS (clk)
BEGIN
  IF (clk'EVENT AND clk='1') THEN
    IF (counter=3) THEN
      c <= '1';
    ELSE
      c <= '0';
    END IF;
  END IF;
END PROCESS;
----Generating the outputs:-----
out1 <= c;
out2 <= (a AND b) OR c;
END signal_generator;

```



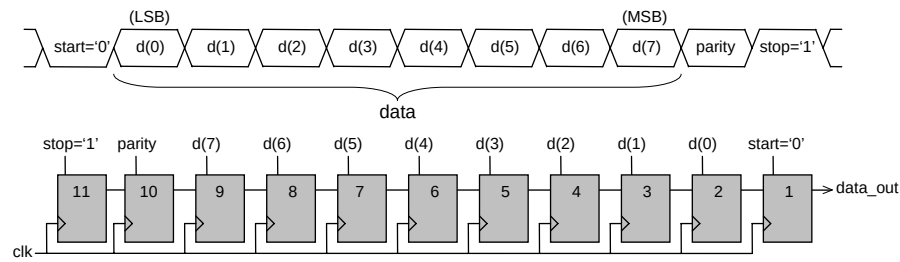
Simulation results for Problem 8.6.

Chapter 9: Additional Circuit Designs

Problem 9.5: Serial data transmitter

Solution:

A typical way of solving this kind of problem is to store the data vector into a shift register (when the data is ready, of course) and then shift the values out sequentially. The general data structure is then that shown in the figure below. Note that the overall protocol has been modified slightly to resemble an actual SPC (Single Parity Check) code, similar, for example, to that used in PS/2 computer keyboards (see chapter 7 of [1]). A corresponding VHDL code is shown below, followed by simulation results.

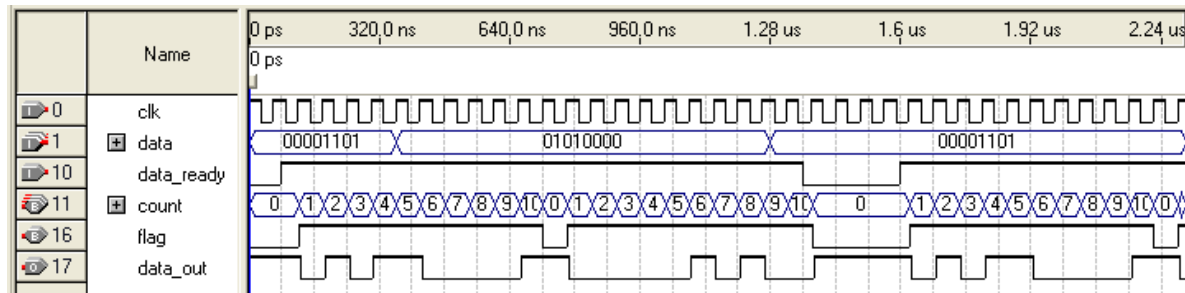


Data structure for Problem 9.5.

```

-----
ENTITY serial_tx IS
  PORT (clk: IN BIT;
        data_ready: IN BOOLEAN;
        data: IN BIT_VECTOR(7 DOWNTO 0);
        data_out: OUT BIT);
END serial_tx;
-----
ARCHITECTURE serial_tx OF serial_tx IS
  SIGNAL shift_register: BIT_VECTOR(11 DOWNTO 1) := (OTHERS=>'1');
  SIGNAL parity: BIT;
BEGIN
  ----Determining parity bit (see Problem 6.8):-----
  PROCESS (data)
    VARIABLE temp: BIT_VECTOR(7 DOWNTO 0);
  BEGIN
    temp(0) := data(0);
    FOR i IN 1 TO 7 LOOP
      temp(i) := data(i) XOR temp(i-1);
    END LOOP;
    parity <= temp(7);
  END PROCESS;
  ----Creating shift register and transmitting data:-----
  PROCESS (clk)
    VARIABLE count: INTEGER RANGE 0 TO 12 := 0;
    VARIABLE flag: BIT;
  BEGIN
    IF (clk'EVENT AND clk='1') THEN
      IF (data_ready AND flag='0') THEN
        flag := '1';
        count := count + 1;
        shift_register <= ('1' & parity & data(7 DOWNTO 0) & '0');
      ELSIF (flag='1') THEN
        shift_register <= ('0' & shift_register(11 DOWNTO 2));
        count := count + 1;
        IF (count=11) THEN
          count := 0;
          flag := '0';
        END IF;
      END IF;
    END IF;
    data_out <= shift_register(1);
  END PROCESS;
END serial_tx;
-----

```



Simulation results for Problem 9.5.

Chapter 10: Packages and Components

Problem 10.2: Carry-ripple adder constructed with components

Solution:

A VHDL code for this exercise is shown below. Further details, including its physical implementation and other adder architectures can be seen in [1] (chapters 12, 19, and 232).

```
-----The component (full-adder unit):-----
ENTITY FAU IS
    PORT (a, b, cin: IN BIT;
          s, cout: OUT BIT);
END FAU;

-----
ARCHITECTURE full_adder OF FAU IS
BEGIN
    s <= a XOR b XOR cin;
    cout <= (a AND b) OR (a AND cin) OR (b AND cin);
END full_adder;

-----
-----Main code:-----
ENTITY carry_ripple_adder IS
    GENERIC (N : INTEGER := 8); --number of bits
    PORT (a, b: IN BIT_VECTOR(N-1 DOWNTO 0);
          cin: IN BIT;
          s: OUT BIT_VECTOR(N-1 DOWNTO 0);
          cout: OUT BIT);
END carry_ripple_adder;

-----
ARCHITECTURE structural OF carry_ripple_adder IS
    SIGNAL carry: BIT_VECTOR(N DOWNTO 0);
    -----
    COMPONENT FAU IS
        PORT (a, b, cin: IN BIT; s, cout: OUT BIT);
    END COMPONENT;
    -----
BEGIN
    carry(0) <= cin;
    generate_adder: FOR i IN a'RANGE GENERATE
        adder: FAU PORT MAP (a(i), b(i), carry(i), s(i), carry(i+1));
    END GENERATE;
    cout <= carry(N);
END structural;
-----
```

Chapter 11: Functions and Procedures

Problem 11.1: Conversion to STD_LOGIC_VECTOR

Solution:

A VHDL code for this exercise is shown below. To test it, just a jumper from the input to the output was used, in which case the compiler must give equations of the type $output(i)=input(i)$, for $i=0, 1, \dots, N-1$. A similar function can be found in the standard package *std_logic_arith*, available in the library of your VHDL synthesis software. Note that in the solution below the function was located in the main code; to install it in a package, just follow the instruction in chapter 11 (see example 11.4, for example).

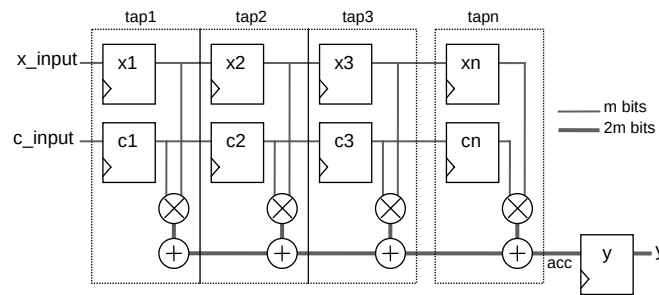
```
-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-----
ENTITY data_converter IS
    GENERIC (N: INTEGER := 4); --number of bits
    PORT (input: IN INTEGER RANGE 0 TO 2**N-1;
          output: OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0));
END data_converter;
-----
ARCHITECTURE data_converter OF data_converter IS
    SIGNAL x: STD_LOGIC_VECTOR(N-1 DOWNT0 0);
    FUNCTION conv_std_logic(arg: INTEGER; size: INTEGER)
        RETURN STD_LOGIC_VECTOR IS
        VARIABLE temp: INTEGER RANGE 0 TO 2**size-1;
        VARIABLE result: STD_LOGIC_VECTOR(size-1 DOWNT0 0);
    BEGIN
        temp := arg;
        FOR i IN result'RANGE LOOP
            IF (temp>=2**i) THEN
                result(i) := '1';
                temp := temp - 2**i;
            ELSE
                result(i) := '0';
            END IF;
        END LOOP;
        RETURN result;
    END conv_std_logic;
BEGIN
    output <= conv_std_logic(input, N);
END data_converter;
-----
```

Chapter 12: Additional System Designs

Problem 12.4: General-purpose FIR filter

Solution:

A block diagram for this circuit is shown below. It contains two shift registers, which store the input values (x) and the filter coefficients (c). It contains also a register to store the accumulated (acc) value, producing the filter output (y). The total number of taps is n , with m bits used to represent x and c , and $2m$ bits for the after-multiplication paths. Hence a total of $2nm+m$ flip-flops are required. Observe that the taps are all alike, so COMPONENT can be used to easily implement this circuit. A slightly more difficult (non-structural) solution is shown below; the structural option (with COMPONENT) is left to the reader. Note that the code includes overflow check. Simulation results are also included.



Circuit diagram for Problem 12.4.

```

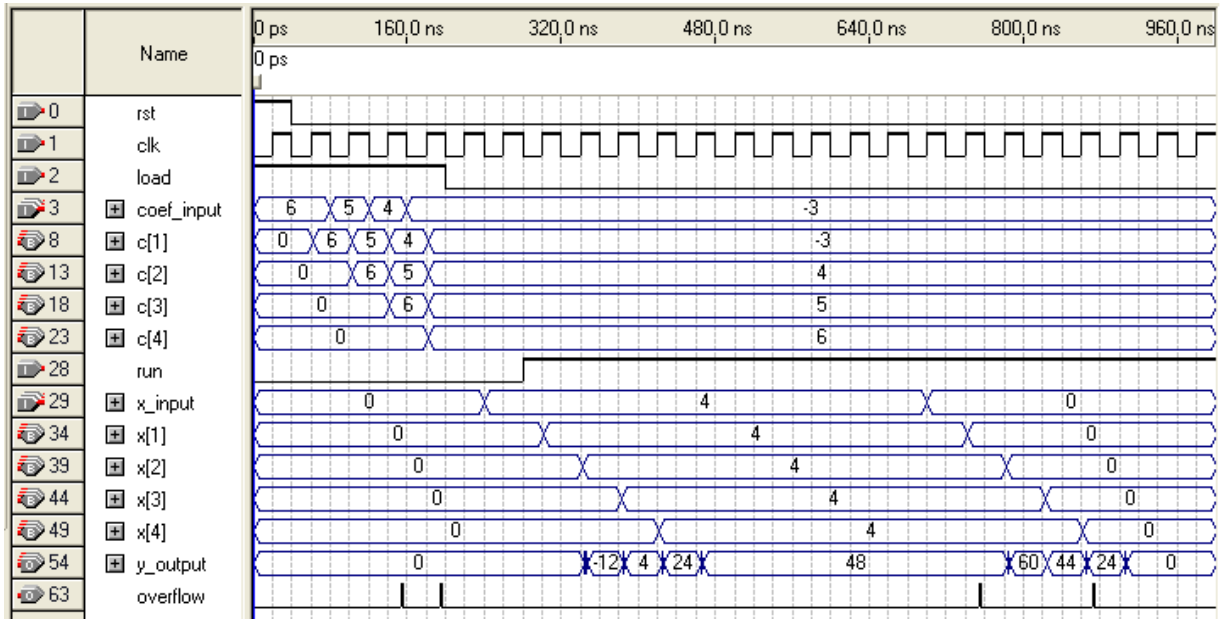
-----
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_arith.all; --package needed for SIGNED
-----
ENTITY FIR IS
    GENERIC (n: INTEGER := 4; --number of coefficients
             m: INTEGER := 4); --number of bits per coefficient
    PORT (clk, rst: IN STD_LOGIC;
          load: STD_LOGIC; --to enter new coefficient values
          run: STD_LOGIC; --to compute the output
          x_input, coef_input: IN SIGNED(m-1 DOWNTO 0);
          y: OUT SIGNED(2*m-1 DOWNTO 0);
          overflow: OUT STD_LOGIC);
END FIR;
-----
ARCHITECTURE FIR OF FIR IS
    TYPE internal_array IS ARRAY (1 TO n) OF SIGNED(m-1 DOWNTO 0);
    SIGNAL c: internal_array; --stored coefficients
    SIGNAL x: internal_array; --stored input values
BEGIN
    PROCESS (clk, rst)
        VARIABLE prod, acc: SIGNED(2*m-1 DOWNTO 0) := (OTHERS=>'0');
        VARIABLE sign_prod, sign_acc: STD_LOGIC;
    BEGIN
        -----Reset:-----
        IF (rst='1') THEN
            FOR i IN 1 TO n LOOP
                FOR j IN m-1 DOWNTO 0 LOOP
                    x(i)(j) <= '0';
                END LOOP;
            END LOOP;
            -----Shift registers:-----
        ELSIF (clk'EVENT AND clk='1') THEN
            IF (load='1') THEN
                c <= (coef_input & c(1 TO n-1));
            ELSIF (run='1') THEN
                x <= (x_input & x(1 TO n-1));
            END IF;
        END IF;
        -----MACs and output (w/ overflow check):-----
        acc := (OTHERS=>'0');
        FOR i IN 1 TO n LOOP
            prod := x(i)*c(i);
            sign_prod := prod(2*m-1);
            sign_acc := acc(2*m-1);
            acc := prod + acc;
            IF (sign_prod=sign_acc AND acc(2*m-1)/=sign_acc) THEN
                overflow <= '1';
            ELSE
                overflow <= '0';
            END IF;
        END LOOP;
    END PROCESS;
END FIR;

```

```

END LOOP;
IF (clk'EVENT AND clk='1') THEN
    y <= acc;
END IF;
END PROCESS;
END FIR;

```



Simulation results for Problem 12.4.